NLU course project: Intent Classification and Slot Filling

Elena Deidda (mat. 267887)

University of Trento

elena.deidda@studenti.unitn.it

1. Introduction

This project compares two strategies for joint intent classification and slot filling on ATIS, evaluated with Intent Accuracy and chunk-level Slot F1 (CoNLL): training a Transformer *from scratch* (Part A) versus fine-tuning pre-trained encoders and decoders (Part B). For Part A, a GPT-2-style decoder-only Transformer is the chosen architecture, with a greedy hyper-parameter search performed over learning rate, architecture, and dropout. For Part B, pre-trained BERT [1] and GPT-2 [2] are used, adapting the sub-tokenization strategy of Chen et al. [3] for slot label alignment. The best model overall is BERT-base, reaching test Slot F1 0.955 and Intent Accuracy 0.975, against 0.888 and 0.934 for the best from-scratch model: pre-trained bidirectional representations give a clear advantage on both tasks.

2. Implementation details

The backbone architectures (GPT-2 from scratch and the fine-tuning setup) follow the course lab; the contributions below are original to this work.

CLS placement (Part A). Since the model is causal, the `CLS` token is appended at the *end* of each utterance rather than at the start. The last position is the only one that has attended to the full sequence, so its hidden state is used for intent classification; slot decoding excludes this extra position before scoring with CoNLL.

Per-architecture intent representation (Part B). BERT uses the `[CLS]` hidden state at position 0. GPT-2 is causal, so intent classification instead uses the hidden state at the last *real* token, retrieved via the per-sequence length rather than the last padded position.

Sub-token label alignment (Part B). Following the strategy discussed in [3], only the first sub-token of each word carries the true slot label; remaining sub-tokens and special tokens are excluded from the slot loss via `ignore_index`. This reference is used as conceptual guidance for the alignment rule, not as a code source.

Multi-seed greedy search. Each candidate in the greedy hyper-parameter search (learning rate first, then architecture, then dropout) is run over multiple seeds *before* being compared to the incumbent, rather than only the final model. This reduces the risk of selecting a configuration that owes its apparent advantage to a favorable seed instead of a real effect of the hyper-parameter.

3. Results

All models are evaluated on the test set over multiple seeds (mean±std); hyper-parameters are selected on the *dev* Slot F1 to avoid test-set leakage. Table 1 and Figure 1 report Part A: the learning-rate search already reaches most of the achievable performance, and only `n_heads` contributes a further, modest gain – every other architecture candidate falls below the incumbent and is discarded. Table 2 reports Part B: BERT clearly outperforms GPT-2 on slot filling (0.955 vs. 0.915 test F1), consistent with BERT's bidirectional context being better suited to per-token tagging than GPT-2's left-to-right attention. Scaling to the large/medium variant does not improve test metrics for either family despite 3× more parameters, suggesting ATIS is already saturated by base-sized pre-trained models. Figure 2 makes this explicit: moving from a from-scratch model to a pre-trained one yields a large jump in Slot F1, while further scaling within Part B yields essentially none. Figure 3 shows the seed-to-seed variability (mean±std) behind every final model in Tables 1–2.

Table 1: *Part A — best model per step (231K param. throughout). Final:* d=64*, 4 heads, 2 layers, FFN 256, dropout 0.*

Step	Adopted	Dev F1	Test Slot F1	Test Int. Acc
0 – lr	0.01	0.929	0.876±0.007	0.921±0.017
1 – architecture	heads=4	**0.941**	**0.888±0.006**	**0.934±0.006**
2 – dropout	none (0)	0.941	0.888±0.006	0.934±0.006

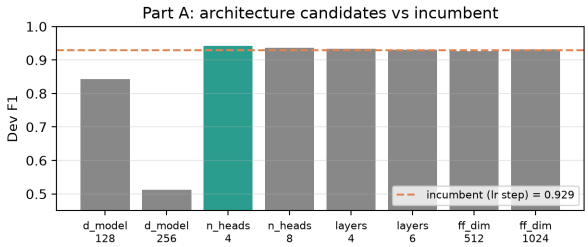

Figure 1: *Part A: dev F1 of every architecture candidate against the incumbent score (dashed line); only* `n_heads`*=4 wins.*

4. Use of AI assistance

I collaborated with Claude (Anthropic) while carrying out this project: it helped set up and debug the remote GPU environment, launch and monitor the training runs, generate the figures, and typeset this report. The choice of hyper-parameters to search, the model architecture, and the interpretation of the results are my own.

5. References

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019.

[2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019.

[3] Q. Chen, Z. Zhuo, and W. Wang, "BERT for joint intent classification and slot filling," in *arXiv preprint arXiv:1902.10909*, 2019.

Table 2: *Part B — fine-tuning (best lr/dropout per family).*

Model	Param	Dev F1	Test Slot F1	Test Int. Acc
BERT-base	109.6M	**0.984**	**0.955±0.002**	**0.975±0.001**
BERT-large	335.3M	0.983	0.955±0.001	0.975±0.002
GPT-2-base	124.6M	0.963	0.915±0.004	0.971±0.002
GPT-2-medium	355.0M	0.964	0.913±0.002	0.969±0.003

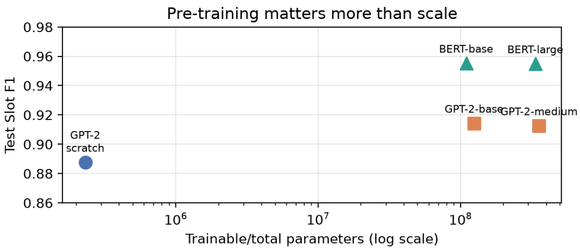

Figure 2: *Test Slot F1 vs. parameter count (log scale): the jump from Part A to Part B dwarfs any gain from scaling within Part B.*

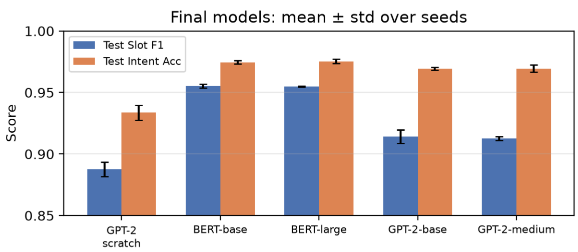

Figure 3: *Final models: test Slot F1 and Intent Accuracy, mean±std over seeds.*